# LayerTree: A Shared Bitcoin UTXO as an Ideal Functionality, and Its Realisation with Covenants

Olaoluwa Osuntokun

Lightning Labs
laolu@lightning.engineering

2026-10-10
Draft (commit a1a493f)

**Abstract.** LayerTree lets many users share one Bitcoin output: the output commits to an off-chain Merkle-sum tree of balances, and tapscript covenants enforce every change to it. Any member exits alone, and the operator evicts a member whose lease has expired, in one transaction of about 1,020 virtual bytes at depth 16, with no pre-signed interior and no expiry of the shared output; a cohort joins, settles or is swept in one transaction. We describe each operation in the order a member meets it, who authorises it and what it costs, and the voucher system: a signature over a source leaf and a new leaf that anyone can redeem once, which an issuer re-signs for a running total, so repeated payments to one recipient, including payments received over Lightning, cost nothing on-chain and never expire. The design compiles to four tiers of proposed opcodes, the smallest needing OP_CAT and one point-tweak opcode; channels live in leaves; the tree rules are mechanised in Lean 4, and a simulator, a byte-exact Go implementation and 2,349 executable checks accompany it.

**Keywords:** Bitcoin · covenants · Merkle-sum trees · vouchers · payment channels · formal verification

## 1 Introduction

Payment channels [29] move Bitcoin activity off-chain for pairs; the open problem is doing it for *groups*: letting $N$ users share one unspent transaction output (UTXO) without giving up the ability of each to leave alone. Most deployed or proposed approaches pay for the sharing when it is dissolved. In channel factories [6], Ark [17,30], timeout trees [18,43] and covenant trees built on OP_CHECKTEMPLATEVERIFY (CTV) [31] or OP_CHECKCONTRACTVERIFY [15,14], the shared output is the root of a tree of *transactions*. A member who leaves unilaterally publishes the path from the root to its leaf, $O(\log N)$ transactions each paying a fee; the interior must exist in advance as pre-signed transactions (every member signs, online) or covenant templates; and when the tree expires,

reclaiming the funds of one unresponsive member means publishing everything. Payment-pool opcode proposals (`OP_TAPLEAF_UPDATE_VERIFY` [37], CoinPool's `OP_MERKLESUB` [21], `OP_EVICT` [42]) target one-transaction withdrawal instead, with purpose-built opcodes.

This paper takes a different route. The tree stays off-chain and is a tree of *data*: a binary Merkle-sum tree of balances whose root one output commits to. A member exits by presenting its leaf and sibling path to the output itself; the script recomputes the root from the proof, checks it against the commitment, recomputes it with the leaf removed, and requires the transaction to recreate the output under the new root with the leaf's amount paid out. The member needs nobody's signature, and the exit is one transaction whose size grows with the tree's depth. The same covenant lets the operator evict members whose lease has expired, graft a pre-built subtree of new members in one step, and sweep an expired cohort cheaply.

This paper explains the design for those who would build on it: the tree (§3), every operation in the order a member meets it (§4), vouchers, which pay a member or receive from Lightning with no one's participation at redemption (§5), and who can do what (§6). The operations compile to four tiers (§7), from restored string opcodes [34,35], introspection [33], stack signature checks [4] and elliptic-curve arithmetic [24,32] down to `OP_CAT` and one point-tweak opcode, plus one on `OP_CHECKCONTRACTVERIFY` [15]. We propose no opcode, but three tiers assume opcodes in a tapleaf version their proposals do not define. The elliptic-curve draft [24] is by the author. The ideal functionality, its UC proof and the Lean development are in the full version (§8). The specification, code and proofs are public [25]; this text describes revision a1a493f.

## 2 Background

*Bitcoin.* A pay-to-taproot output [39] commits to an internal key $P$ and the Merkle root $m$ of a *taptree* of scripts through $Q = P + H_{\mathsf{TapTweak}}(P \,\|\, m) \cdot G$, and is spent either by a Schnorr signature [38] under $Q$ or by revealing one script with its Merkle path (the *control block*) and satisfying it. Tapscript [40] has no loops, so every "loop" below is unrolled. BIP-441 [35] defines tapleaf version `0xC2`, restoring the string opcodes disabled in 2010 (`OP_CAT`, `OP_SUBSTR`, `OP_LEFT`, `OP_RIGHT`, bit operations) with unsigned numbers of any length, metered by BIP-440 [34] ("varops"); BIP-347 [13] proposes `OP_CAT` alone in the current version, `0xC0`. `OP_TX` [33] pushes fields of the spending transaction, optionally collated into one hash preimage; `OP_CHECKSIGFROMSTACK` (CSFS) [4] verifies a signature over a stack element; `OP_TWEAKADD` [32] computes $x(P + tG)$, and the EC-ops draft [24] adds point arithmetic. All are proposals. Sizes are in virtual bytes (vB), a quarter of a transaction's weight in weight units (WU), where a witness byte weighs one unit and any other byte four. A Merkle-sum tree [19,23] hashes each node's children with their sums, so an inclusion proof proves a leaf's contribution to the root sum.

## 3 How LayerTree Works

A *tree* has one *operator* and any number of *members*. Each member owns a *leaf*: a balance and three keys (an owner key that authorises the member's actions, an exit key that an exit pays, and an evictor key, normally the operator's, that may remove the leaf once its lease height has passed). The operator holds no key over the funds, only an *operator key* that admits new members and co-signs cooperative changes. The balances live in a Merkle-sum tree kept off-chain; one taproot output holds their total and commits to the root, and each operation is a tapscript leaf of that output, which we call its *predicate*.

### 3.1 The Tree

We write $\mathsf{le}_k(x)$ for the $k$-byte little-endian encoding of $x$. A leaf is a fixed 116-byte record

$$\mathsf{le}_{64}(\mathsf{amount}) \,\|\, \mathsf{exit_key} \,\|\, \mathsf{owner_key} \,\|\, \mathsf{evictor_key} \,\|\, \mathsf{le}_{32}(\mathsf{evict_after}) \,\|\, \mathsf{le}_{32}(\mathsf{nonce}) \,\|\, \mathsf{le}_{32}(\mathsf{seq}),$$

with keys as 32-byte x-only points, $\mathsf{evict_after}$ a block height, $\mathsf{nonce}$ a per-leaf identifier and $\mathsf{seq}$ a counter advanced by vouchers (§5). A *node* is a pair $(h, s)$ of a 32-byte hash and an unsigned sum. A position holding leaf $\ell$ has node $(H_{\mathsf{leaf}}(\ell), \ell.\mathsf{amount})$; an empty position has node $E_0 = (0^{32}, 0)$. The parent of $(h_L, s_L)$ and $(h_R, s_R)$ is

$$\big(H_{\mathsf{node}}(h_L \,\|\, \mathsf{enc}_{64}(s_L) \,\|\, h_R \,\|\, \mathsf{enc}_{64}(s_R)),\ s_L + s_R\big), \qquad \mathsf{enc}_{64}(s) = \mathsf{le}_{64}(s) \,\|\, \mathtt{0x01}.$$

Every hash is a BIP-340 tagged hash with its own tag; $\mathsf{enc}_{64}(s)$ is the minimal BIP-441 encoding of $s + 2^{64}$, produced from a computed sum with one addition. $E_h$ is the node of the empty tree of height $h$.

The *sibling path* $\pi$ of a position is the list of $D$ pairs $(\mathsf{dir}_k, (h_k, s_k))$, one per level, giving the side the position lies on and the sibling node not taken. $\mathsf{fold}(n, \pi)$ combines a node with a path from the bottom up, hashing with each sibling in the order the direction gives and adding the sums. The honest path folds a leaf's node to the root, and folding a *different* node up the *same* path yields the root with that position replaced (§8).

*Profiles.* In the *dense* profile the operator assigns positions and a path carries direction bits. In the *sparse* profile a leaf's position is derived by hashing its owner key and nonce, as in the Merkle-sum sparse Merkle tree of Taproot Assets [23], truncated to $D = 32$ or 64, at 1.7 to 2.8 times the dense exit cost. Dense is the default and is what we measure.

### 3.2 The Tree Output

Let $r$ be the root hash and $S$ the root sum. The tree output is a taproot output with internal key $P = \mathsf{lift_x}\big(x(\mathsf{NUMS} + H_{\mathsf{state}}(r) \cdot G)\big)$, where $\mathsf{NUMS}$ is the BIP-341 nothing-up-my-sleeve point and $\mathsf{lift_x}$ the BIP-340 even-$y$ normalisation, so

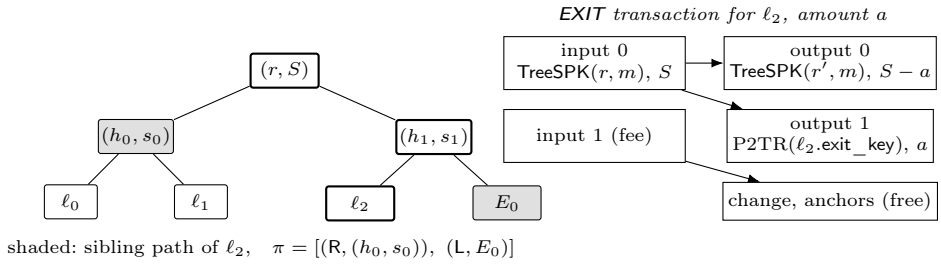



**Fig. 1.** A depth-2 tree and the exit of $\ell_2$. The predicate checks $\mathsf{fold}((H_{\mathsf{leaf}}(\ell_2), a), \pi) = (r, S)$, computes $r' = \mathsf{fold}(E_0, \pi)$, verifies a signature by $\ell_2.\mathsf{owner_key}$, and constrains outputs 0 and 1. Everything else in the transaction is free.

the key path is unspendable and every spend goes through a script. Its taptree has one leaf per predicate; its Merkle root $m$ identifies the rule set and is fixed for the tree's life. A *tree output* for state $(r, S)$ has scriptPubKey $\mathsf{TreeSPK}(r, m)$ and amount $S$. Every predicate recomputes $\mathsf{TreeSPK}(r, m)$ from its witness and checks it against the spent input, binding the root the proofs are checked against and the script set the successor carries. FULL, MID and MIN predicates read input 0, so two such trees cannot be spent in one transaction; a CCV tree's input may sit at any index.

## 4 Operations

Every operation is one spend of the tree output through one tapleaf. The tree output is input 0, and output 0 recreates it under the new root and total. Payouts are outputs $1, \ldots, K$, each paying exactly a removed leaf's amount to its exit key; every other input and output is free, for fees, change and anchors. The witness proves each changed position with a *step*: the old node folded up its sibling path must give the committed $(r, S)$, and the new node folded up the same path gives $(r', S')$; steps chain. Signatures cover all outputs. If the last leaf leaves, output 0 is unconstrained. Costs are whole transactions at $D = 16$ in FULL, on the model of §10.

### 4.1 Joining

*JOIN$_h$: grafting a cohort.* The operator inserts a subtree of up to $2^h$ new leaves, built off-chain, into an empty aligned slot of height $h$; $h = 0$ adds one leaf. *Authorised by* the operator key alone; no existing member signs or is affected. *Proves* that the slot held the empty node $E_h$, and adds the new subtree's root and sum. *On chain* the tree output grows by the subtree's sum, which other inputs must supply. The covenant sees a hash and a sum, not the leaves, and walks only the $D - h$ levels above the slot, so a larger cohort is a smaller transaction: 937 vB

for one leaf, 872 vB for four (218 each), 679 vB for 256 (2.7 each). A taptree carries one $\mathsf{JOIN}_h$ leaf per height it supports.

Because the covenant cannot look inside the graft, each joiner checks it. A member funding its own leaf verifies, before signing its input over all outputs, that the slot folds as empty to the current root, that its leaf folds to the grafted node with every sum recomputed, and that output 0 commits to the new root; its input can then only be spent into a root containing its leaf. A self-funded cohort is thus a signing round, like an Ark round or a coinjoin, and each member adds its funding input (58 vB). In an operator-funded graft the member pays off-chain, after checking its path against the confirmed root, or through a hash-locked voucher (§5.4).

*$\mathsf{JOINK}_K$: batched single joins.* $K$ single-leaf joins in sequence under one operator signature. It is the batching primitive of the sparse profile, where a leaf's position is derived from its key and cannot be grafted as part of a subtree, and costs 1,494 vB for two in the dense profile.

### 4.2 Moving Value

Most payments never touch the tree. Members with channel leaves pay over Lightning through the operator (§9), and vouchers (§5) move value with a signature that is redeemed only if the issuer stops cooperating. The tree changes when a balance's terms change or a carve is redeemed.

*UPDATE and $\mathsf{UPDATE}^x$: changing a leaf.* Replaces one leaf with another at the same position: a splice in or out, a lease renewal, a key rotation or a transfer of ownership. *Authorised by* the leaf's owner and the operator together; $\mathsf{UPDATE}^x$, a variant chosen at creation, adds the leaf's exit key as a third signer. *Proves* the old leaf against the root and writes the new one. *On chain* the tree output changes by the difference: an increase is funded by other inputs, a decrease is free for the transaction to spend, and both signers see where it goes. 1,030 vB, or 1,048 vB with the third signature. $\mathsf{UPDATE}^x$ exists for leaves whose exit key is shared with or owned by someone else (a channel's counterparty, a credit leaf's recipient, §5.3): without it the owner and the operator could splice such a leaf down together.

*SPLIT: paying a member directly.* Carves a new leaf out of the payer's leaf. *Authorised by* the payer's owner key, signing this transaction. *Proves* two positions in sequence: the source leaf, rewritten with its amount reduced by the payment and its seq raised by one (or emptied if drained), then an empty position under the intermediate root, which receives the new leaf. *On chain* the total is unchanged and no payout is made. The new leaf keeps the source's evictor key and a lease no later than the source's, and neither it nor a non-zero residual may be below DUST_MIN (both rules are shared with CLAIM, §5.1). 1,586 vB. SPLIT is the right shape when the payer is the one broadcasting and paying the fee; it leaves no outstanding promise behind, and it is the one member-to-member transfer

the MIN tier supports. Its deferred counterpart, a voucher redeemed by CLAIM, is the subject of §5.

### 4.3 Settling a Cohort

*REWRITE$_h$: everything at once.* Replaces the contents of one aligned subtree of height $h$ in a single spend: any position may go from a leaf to empty (an exit, paid out), from empty to a leaf (a join, funded) or from one leaf to another (a payment, renewal, resize or key rotation). *Authorised by* the operator and by the owner of every leaf that changes; a leaf that only gains amount needs no signature unless its seq is SEQ_FINAL (a channel leaf, §9, whose capacity its signed states fix), so a payment within a cohort needs only the payer's, and a leaf nobody signs for passes through unchanged. *Proves* the old subtree, every leaf of it published, against the root, and computes the new one. *On chain* each emptied leaf is paid exactly its amount at its exit key, so the operator cannot use the rewrite to remove a leaf without its owner's signature and its payout. At $h = 6$ a mass exit costs 161 vB per leaf, about what a sweep costs the operator, and rewriting every balance 147 vB per leaf against 1,030 vB for an UPDATE each. REWRITE$^x_h$ adds the exit key's signature (16 vB per signed position) and replaces REWRITE$_h$ wherever UPDATE$^x$ or CLAIM$^x$ is present. FULL only.

### 4.4 Leaving

*EXIT$_K$: unilateral exit.* Removes one leaf, or $K$ leaves in one transaction, and pays each out (Figure 1). *Authorised by* the leaf's owner key alone; no operator, no other member, no timelock. *Proves* the leaf and its path against the root and folds the empty node up the same path for the new root. *On chain* the tree output shrinks by the leaf's amount and output 1 pays its exit key. In FULL the payout may be less than the balance, the difference paying the fee, which saves the fee input (99 vB); the exact-payout variant EXIT$^=_K$, chosen at creation and visible in the tree's taptree root, requires the full amount and is what MID, MIN, CCV and channel leaves use. 1,020 vB for one exit, and 665 vB more for a second in the same transaction (EXIT$_2$ is 1,684 vB). The owner's signature commits to the tree outpoint, which every operation changes, so an exit cannot be pre-signed: the owner must be online to rebuild it after a competing spend. Owners first ask for a cooperative exit, which the operator can batch.

### 4.5 Being Evicted

*Eviction by EXIT.* A lease is how the operator recovers its capital and slots from a member who has gone away, and eviction is the only way it can remove a leaf without the owner. The exit script, with a flag, removes a leaf whose lease has passed. *Authorised by* the leaf's evictor key, with a lock time at or after the lease height. *On chain* the payout is exactly the leaf's amount, to the leaf's own exit key, so an eviction returns the member's money rather than taking it. Same cost as an exit; the evictor pays the fee.

*EVICT_SUBTREE$_h$: the cohort sweep.* Evicts every remaining leaf of one aligned subtree in one spend. *Authorised by* the operator key, which must be every swept leaf's evictor, with a lock time past every lease. *Proves* the subtree from all its leaves (empty positions contribute $E_0$), each internal node hashed once, against the root. *On chain* one exact payout per leaf, and the subtree becomes empty. 203 vB per leaf for 13 leaves at $h = 4$ and 158 vB per leaf for 51 at $h = 6$, against 1,020 vB each alone. One leaf with a later lease holds up its cohort's sweep until that lease passes, and one with another evictor blocks it, so operators admit only leaves that name the operator as evictor into a cohort, align leases per cohort, and renew a member by moving it to a cohort with the new lease (an UPDATE followed by a SPLIT of the whole balance).

### 4.6 Scaling Out

*DETACH$_h$: a cohort's own output.* Moves one aligned subtree into a tree output of its own. *Authorised by* the operator key. *Proves* the subtree's root and sum against the parent root. *On chain* output 0 is the parent minus the cohort and output 1 the cohort, under its own root and the parent's rules compiled at depth $h$; the detached tree inherits the parent's tree identifier, so vouchers and channel states against its leaves stay valid. No member needs new data: its proof in the detached tree is its old path cut to the lower $h$ levels. 780 vB at $h = 8$, after which every exit from the cohort costs 762 vB instead of 1,020; the detach pays for itself at the fourth exit. Its larger effect is throughput, since each output is spent independently (§6).

## 5 Vouchers

A voucher lets a member pay someone, or the operator pay a member who receives from Lightning, with a signature alone. Nothing reaches the chain when it is issued, and its holder can redeem it later without the issuer or the operator. Vouchers are the LayerTree counterpart of Ark's out-of-round payments, with one difference: a redeemed voucher is a leaf of the tree, which its holder can exit alone, not an output that expires with a round.

### 5.1 The Voucher and CLAIM

A voucher is a BIP-340 signature by the owner key of a *source* leaf over $H_{\mathsf{voucher}}(\mathsf{tree_id} \,\|\, \ell_{\mathrm{src}} \,\|\, \ell_{\mathrm{claim}})$, where $\ell_{\mathrm{src}}$ and $\ell_{\mathrm{claim}}$ are full 116-byte leaves and tree_id is a 32-byte constant fixed when the tree is created. $\ell_{\mathrm{claim}}$ is the leaf being paid: an amount and the recipient's keys. Anyone who holds the voucher can redeem it with CLAIM, which checks the signature with `OP_CHECKSIGFROMSTACK` and needs no other. Like SPLIT, it is two steps (Figure 2). The first proves $\ell_{\mathrm{src}}$ against the committed root and rewrites it with its amount reduced by the claim's and its seq raised by one, or empties the position if the voucher draws the whole

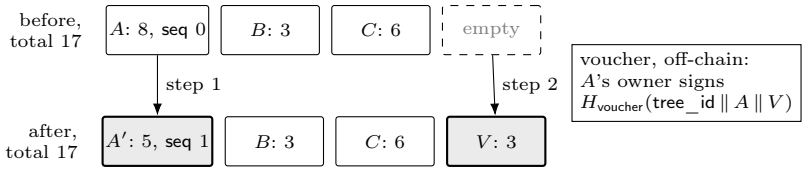



**Fig. 2.** Redeeming a voucher for 3 from $A$ to $V$. CLAIM shrinks the source and advances its seq, then fills an empty position with the claim leaf, with no signature but the voucher's. The tree now holds $A'$, not $A$, so the signed message never matches again.

balance. The second proves a position empty under the intermediate root and places $\ell_{\text{claim}}$ there. The total is unchanged.

Two rules bind every carve, by voucher or by SPLIT. The claim leaf keeps the source's evictor key and a lease no later than the source's, so moving a balance cannot escape eviction, nor plant a leaf in a cohort that the operator could never sweep. And neither the claim leaf nor a non-zero residual may be below DUST_MIN, by default 330 sat, the P2TR dust threshold: a smaller leaf could leave only through a payout that default policy does not relay, so a one-satoshi carve could block its cohort's sweeps and grafts for the life of the tree.

*Single use.* Replay is impossible because redemption changes the leaf the voucher names, and seq keeps it changed even if the balance later returns to its old value. Two rules that no script enforces carry the rest: a leaf's seq only ever advances (an UPDATE winding it back would revive old vouchers), and no two leaves share an owner key and nonce, since a voucher binds its source by content, not position. Binding tree_id rather than the taptree root $m$ keeps a voucher out of other trees built with the same parameters, and since a detached cohort inherits tree_id, vouchers survive a $\mathsf{DETACH}_h$.

### 5.2 Cumulative Vouchers

Every voucher against a source draws on the same leaf, and the first redemption voids the rest. An issuer that pays one recipient repeatedly therefore signs a *cumulative* voucher for the running total against the same claim leaf, and re-signs it for a larger amount at each payment: 2, then 5, then 9. Each payment is one signature off-chain, with no on-chain cost, no minimum and no expiry. The recipient keeps only the latest voucher and, if the issuer stops cooperating, redeems that one alone; it never prefers a smaller one, so it needs no window in which to contest an old state.

Settled credit is retired by advancing seq. Once the issuer pays the recipient another way, for instance with a leaf of the recipient's own in the next cohort graft (under 3 vB per member), it advances the source's seq, which voids every outstanding voucher against it: one UPDATE per source leaf (1,030 vB), or one $\mathsf{REWRITE}_h$ for a cohort of them (about 147 vB per leaf at $h = 6$). Until then the issuer tracks every voucher it has signed as a liability. Plain CLAIM leaves a hole:

anyone holding an old voucher can redeem it, and the issuer holds them all, so redeeming the voucher for 2 voids the one for 9. The credit leaf below closes it.

### 5.3 What the Holder Trusts

The covenant prevents over-*redemption*: the first redeemer is paid by consensus. It cannot prevent over-*issuance*, or voiding: the issuer can advance seq first, by CLAIM, SPLIT or UPDATE. So the holder of an ordinary voucher extends credit to the issuer until it redeems or is paid. Eviction voids vouchers too: once the source's lease passes, its evictor can pay the whole balance to the source's exit key, normally the issuer's. A holder therefore treats a voucher whose source lease can pass before a redemption would confirm (within $\Delta(q)$, §6) as unsecured. A partial-balance voucher also needs an empty position for its claim leaf, which the operator controls, so a holder prefers a full-balance voucher.

*The dedicated credit leaf.* A dedicated credit leaf is a source leaf that the issuer owns and whose exit key is one recipient's, in a tree whose taptree carries $\mathsf{CLAIM}^x$, $\mathsf{UPDATE}^x$ and the exact-payout exit, carries $\mathsf{REWRITE}^x_h$ in place of $\mathsf{REWRITE}_h$ (or no rewrite at all), and omits SPLIT. $\mathsf{CLAIM}^x$ is CLAIM with one further signature over the transaction, by the source leaf's exit key: 7 bytes of script and 64 of witness in FULL, the only tier that builds it. The recipient's own redemptions carry that signature at no cost, and the issuer cannot carve the leaf at all, not even with a voucher it signs to itself. Over-issuance then has no second victim, since every voucher pays the same recipient, who can watch the leaf's amount. Voiding needs the leaf to change, and every transition that could change it needs the exit key: $\mathsf{CLAIM}^x$, $\mathsf{UPDATE}^x$ and $\mathsf{REWRITE}^x_h$ by rule, while the exact exit and an eviction pay the whole balance to that key and so void nothing the recipient is owed. The balance can only grow or pay its recipient, whose remaining exposure is the issuer's liveness: a provider that stops issuing.

Three details keep it that way. The recipient checks from the taptree root $m$ that no plain $\mathsf{REWRITE}_h$ is present, since it needs only the owner's and the operator's signatures, both the issuer's. The issuer funds the leaf only with credit it has issued or is about to, since any excess is the recipient's on every path the issuer can take alone, and tops it up by an amount-only increase under $\mathsf{REWRITE}^x_h$, which needs only the operator's signature. And the leaf has a finite lease with the operator as evictor, not a NUMS evictor, which every carve would inherit, blocking the sweep of whatever cohort the claim leaf lands in. The credit is one-way: if the recipient could also spend from it, an older, larger voucher would be a stale state needing a window to supersede, as in a bidirectional channel. To spend, the recipient redeems into a channel leaf (§9).

### 5.4 Hash-Locked Vouchers and Swaps

$\mathsf{CLAIM}^h$ adds a payment hash to the signed message, under its own tag so that the lock cannot be stripped, and requires the 32-byte preimage: the script hashes it into the message, so a wrong preimage fails as a bad signature. The same

hash can sit in a channel-leaf HTLC or a Lightning invoice, but the pairing is an ordered exchange, not an atomic swap: the issuer can void the voucher until the redemption confirms, and a voucher has no deadline, since script can forbid a spend before a height but not after one. Five rules (spec 9.6.2) order it. (1) The party the HTLC pays draws the preimage. (2) The voucher resolves second; no HTLC waits on a preimage that only a redemption would reveal. (3) The issuer leaves the source's seq alone from the moment an HTLC on the hash is offered to it until the voucher is redeemed or the HTLC fails. (4) No script enforces rule 3, so the holder redeems as soon as it learns the preimage and bears the issuer risk until that confirms. (5) The holder first checks that the source's lease outlasts the HTLC's expiry plus $\Delta(q)$. Usually the HTLC settles off-chain, so such a swap costs at most one $\mathsf{CLAIM}^h$; it is also how a newcomer buys a leaf with Lightning funds.

### 5.5 Receiving over Lightning

*The flow.* A user with no on-chain funds receives a Lightning payment into a credit leaf that the operator owns and that names the user's key as exit key. The payer's HTLC reaches the operator, which obtains the preimage and settles that inbound HTLC *first*; only then does it sign the user a voucher for the new running total, a plain cumulative voucher against the credit leaf, redeemable under $\mathsf{CLAIM}^x$. Nothing reaches the chain, there is no minimum per payment beyond DUST_MIN at redemption, nothing expires, and the recipient need not be online when it is paid. The order matters. A hash-locked voucher issued before the HTLC settles, with the preimage held by the recipient, breaks the rule that the party the HTLC pays draws the preimage (§5.4), and on a credit leaf it cannot be repaired: if the HTLC then fails, retiring the voucher means advancing the credit leaf's seq, which needs the recipient's exit key, so the voucher stays redeemable for a payment that never arrived. Settling first leaves the recipient exposed for one payment, the one the operator has settled and not yet vouched for.

*Settlement.* Normally the voucher is never redeemed. The operator places a leaf of the recipient's own in its next cohort graft, at under 3 vB per member in a 256-member $\mathsf{JOIN}_8$, and retires the voucher by advancing the credit leaf's seq and lowering its amount by what it has paid out: one $\mathsf{UPDATE}^x$ per credit leaf (1,048 vB), which the recipient co-signs, or one $\mathsf{REWRITE}^x_h$ for a cohort of credit leaves, about 147 vB per changed leaf at $h = 6$ plus 16 vB for each exit-key signature. If the operator stops cooperating, the recipient redeems its latest voucher alone with one $\mathsf{CLAIM}^x$: 1,635 vB at $D = 16$, about 16,350 sat at 10 sat/vB, with no timelock, against 1,311 to 2,511 vB and a relative timelock to unroll an Ark round tree of 256 to 65,536 members (§10).

*Costs.* Issuing a voucher costs nothing on-chain. A redemption proves two positions, so at $D = 16$ it costs more than an exit: 1,617 vB for CLAIM, 1,627 vB for $\mathsf{CLAIM}^h$ and 1,635 vB for $\mathsf{CLAIM}^x$, against 1,586 vB for SPLIT, on the model of

**Table 1.** Who can do what (spec 10.1). "Carve" is SPLIT or a redeemed voucher.

| party | can | cannot |
|---|---|---|
| Operator | refuse to co-sign; graft leaves, including ones it funds; evict a leaf after its lease, paying it in full, with everything carved from it; issue vouchers on its own leaves; detach a cohort; fill a cohort's empty positions, stalling partial-balance vouchers | move or reduce another leaf; delay an exit beyond [C] (it contends by fee, like anyone); freeze value it did not graft and fund; change the taptree, a lease, or a detached cohort's rules, keys or balances |
| Owner | exit at any time; co-sign updates; issue vouchers against its leaf; carve its leaf | affect any other leaf; withdraw more than its amount; carve a leaf with another evictor or a later lease than its own |
| Anyone | redeem a valid voucher; relay transitions | trigger an owner's exit; create leaves; alter amounts |

§10, which charges a carve no payout output (with one, add 43 vB). Redemption is the fallback; normally vouchers are re-signed off-chain and retired in batches.

## 6 Trust Model

Custody never rests with the operator. The tree output's key path is unspendable, and every script path needs either a proof against the committed root plus the affected owner's signature, or the operator's signature for operations that only add value or move a cohort intact. Table 1 lists what each party can and cannot do.

*What a member does.* A member stores its leaf and path and follows every spend of the tree output; each witness carries the path it used, so the member updates its own path from the chain alone and needs nothing else to exit. It verifies its path before funding a join (§4.1) and the source before accepting a voucher (§5.3), and must be online to exit, since its signature commits to the current outpoint.

*Liveness.* Every operation spends and recreates one output, so an honest exit can be overtaken by conflicting spends (a corrupt member alone can carve its balance back and forth), and its delay depends on policy and fees. We state it as an assumption. *[C] (contention):* at most $\kappa \geq 1$ spends of a tree output confirm per block, and while an honest spend is pending at the prevailing feerate at most $\beta$ conflicting adversarial spends confirm ahead of it. $\kappa$ is mempool policy (25 under Bitcoin Core's chain limits before version 31, at most 64 under its cluster limit since [3,9], as low as 1 once anyone hangs descendants off a payout); $\beta$ is the adversary's fee budget against the honest bid. Under [C] an honest spend with $q$ others queued ahead confirms within $\Delta(q) = \Delta_L(\lceil(\beta + q)/\kappa\rceil + 1)$ blocks, $\Delta_L$ being the chain's own delay. Cooperatively a tree drains 25 exits per block, so 10,000 leaves closing at once take 400 blocks: a tree is sized by exit throughput, not depth. The operator can evict any leaf once its lease passes, so a member who wants a longer guarantee needs a later lease at entry; no carve extends one.

**Table 2.** The four opcode tiers. "Shape" says whether inputs and outputs the predicate does not constrain are free or fixed by the script. Sizes are bytes of script / witness data for $\mathsf{EXIT}_1$ in the dense profile at $D = 16$; the last row is the transaction size on the model of §10. MID's sweep fits at most 54 occupied positions at $h = 6$ (stack limit).

| | **FULL** | **MID** | **MIN** | **CCV** |
|---|---|---|---|---|
| Tapleaf version | `0xC2` | `0xC0` | `0xC0` | `0xC2` or `0xC0` |
| Beyond BIP-342 | `OP_CAT`, splice, bit ops, `OP_TX`, CSFS, EC ops | `OP_CAT`, `OP_TX`, CSFS, `OP_TWEAKADD` | `OP_CAT`, `OP_TWEAKADD` | `OP_CCV` (BIP-443), `OP_TX` |
| Numerics | any length | 4-byte, $< 2^{31}$ | 4-byte, $< 2^{31}$ | as FULL or MID |
| Owner payout | $\leq$ amount | exact | exact | exact |
| Predicates | all | all | $\mathsf{EXIT}_1$, SPLIT | all |
| Vouchers / channels / shape | yes / yes / free | yes / yes / free | no / no / fixed | option / yes / free |
| $\mathsf{EXIT}_1$ script / witness (B) | 2,101 / 902 | 3,367 / 978 | 3,640 / 1,175 | 1,716 / 838 |
| $\mathsf{EXIT}_1$ transaction (vB) | 1,020 | 1,355 | 1,473 | 908 |

## 7 Realisations by Opcode Tier

Each operation of §4 is compiled to tapscript against four opcode sets (Table 2), so that the design's dependence on each candidate opcode is measured rather than asserted; the tree format and the predicates are the same in every tier. FULL, MID and CCV compile every operation, MIN the dense single-leaf exit and SPLIT; $\mathsf{CLAIM}^x$, $\mathsf{REWRITE}_h$ and $\mathsf{DETACH}_h$ are FULL's alone. Table 3 (Appendix A) lists what each tier assumes of the proposals it uses, mirroring the assumptions [A-1] to [A-6] of the specification. MIN is the tier closest to the proposals as written: it needs only `OP_CAT` (BIP-347) and `OP_TWEAKADD` (BIP-449), both defined for the current tapleaf version. MID and CCV in `0xC0` need `OP_TX` in a version its draft does not define; CCV in `0xC2` needs BIP-443 there; FULL needs CSFS and EC ops in `0xC2` and two drafts whose opcode numbers collide.

*What each tier gives up.* In FULL the leaf is one stack element and sums use arbitrary-length arithmetic. MID's 4-byte numbers cap a tree at $2^{31}$ satoshis (21.47 BTC) and force exact payouts. MIN has no `OP_TX`: the script builds the BIP-341 signature message from the values it has verified and checks it with a signature under the generator $G$ (Poelstra's construction [27]), which passes only if the message equals the real transaction's (assumption [MIN]); vouchers and channels, which need a signature over stack data, are out of its reach, but SPLIT is not (6,651 bytes of script). CCV replaces every taproot-key recomputation with an `OP_CHECKCONTRACTVERIFY` call and is the smallest tier.

## 8 What Is Proved

The rules of every operation are mechanised in Lean 4 [20] (about 3,400 lines, no Mathlib), with an axiom audit of 60 headline theorems, none resting on an unproved step. They show that a root binds its tree, so a leaf whose path verifies is the leaf at that position; that every sequence of steps conserves the total, with no hash assumption; that every known leaf can be paid out by its honest witness;

that a graft without a real preimage freezes only the value it brought; that a voucher cannot be replayed and a carve keeps the evictor, lease and dust floor; and that the cohort operations and encodings behave as the scripts assume. Sums are conserved unconditionally, but an exit is guaranteed only to a member who verified its path. Scripts are matched to the rules by executable checks (§10), not a verified compiler.

For each tier, the protocol UC-realises an ideal functionality of the shared output in a global-ledger hybrid model [7,2], with idealised hashes, taproot binding in the random-oracle model and [C], against a static adversary corrupting the operator and any members. The full version gives the functionality, the simulator, the proof and the Lean statements.

## 9 Channel Leaves

A leaf whose exit key is a 2-of-2 between its owner and a counterparty (normally the operator) hosts a payment channel whose funding output appears only when an exit or eviction pays the leaf out, so a tree of such leaves is a Lightning hub behind one UTXO. That outpoint changes with every tree transition, so states are signed without committing to it (`OP_TX` collates the signed fields and CSFS checks the signature, emulating `SIGHASH_ANYPREVOUT` [11]), with the eltoo ratchet [10,36], zero-fee version-3 transactions and an ephemeral anchor [41]. This needs FULL, MID or CCV.

*Deadlines and hub size.* Settlement outputs include one per in-flight hashed time-locked contract (HTLC). A forwarding node keeps $\mathsf{cltv_expiry_delta}$ blocks between an incoming and an outgoing HTLC's expiry, and a channel leaf needs $\mathsf{cltv_expiry_delta} \geq G_{\text{exit}}+3+\mathsf{csv_delay}+G_{\text{sweep}}$, with $\mathsf{csv_delay}$ the settlement's relative delay, $G_{\text{sweep}}$ the time to confirm an HTLC sweep and $G_{\text{exit}}$ the time to confirm the exit, $\Delta(q)$ under [C]. With $\mathsf{csv_delay} = 144$, $G_{\text{sweep}} = 2$ and $G_{\text{exit}} = 72$ this is about 220 blocks, against 40 to 80 on the public network. At $\Delta_L = 1$, $G_{\text{exit}} = 72$ needs $\beta + q \leq 71\kappa$; with $\kappa = 25$ and $\beta = 0$, about 1,775 channel leaves may close in one window, and larger hubs shard. The counterparty is normally the operator, which can issue competing spends, so the budget holds only while the member can outbid them, which is what $\beta$ assumes. MID allows seven HTLCs per state (the 520-byte element limit) and $2^{31}$ satoshis per tree.

*Conditions.* A channel leaf has exact payouts, so the owner cannot underfund the channel; $\mathsf{seq} = \mathsf{SEQ_FINAL}$, so no voucher or $\mathsf{SPLIT}$ can carve the capacity the signed states commit to; and lease zero with the counterparty as evictor, so eviction is the counterparty's force-close. The state machine and the budget run end to end on the simulator's chain model with two blocks of margin; a proof that channel leaves compose is open. A force-close costs about 1,681 vB, against about 680 vB for a simple taproot channel [26].

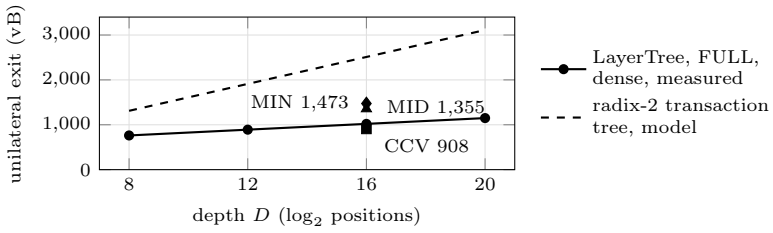



**Fig. 3.** Unilateral exit cost against tree depth: measured for LayerTree (FULL, and the other tiers at $D = 16$), modelled for an unamortised radix-2 transaction tree.

## 10 Evaluation

*Implementation.* A Python tapscript interpreter for both leaf versions computes the real BIP-341 signature message and runs every compiled script against a reference tree, with positive and negative cases per rule (forged leaf, stale root, unconserved amount, early eviction, voucher replay, and others); a Go implementation on `btcd` and `lnd` generates the same scripts byte for byte and builds, signs and verifies every FULL transition.

*The transaction model.* Every vB figure is a whole transaction: the measured script, witness and a 129-byte control block (an eight-leaf taptree), plus a 946 WU skeleton (the tree input, a fee input, the tree output, one P2TR payout and a change output) and 172 WU per further payout; transitions without a payout are charged 172 WU less. Witness length prefixes are excluded, about 1% in FULL and 3% in MID and MIN.

*Exit cost.* A dense exit (Figure 3, Table 4) costs 762 vB at $D = 8$ and 1,149 vB at $D = 20$, about 32 vB per level. One tier step costs as much as 10 (MID, +33%) to 14 (MIN, +44%) levels of depth. The sparse profile at $D = 32$ with $2^{16}$ members costs about 1,780 vB, 1.75 times the dense profile at $D = 16$; its witness grows with $\log_2 N$, not $D$.

*Comparison with transaction trees.* The dashed line of Figure 3 is a *model*: an exiter publishes $D$ transactions of about 150 vB (one key-path input, two P2TR outputs) and a 111 vB claim, anchors and their fee children omitted, which favours the tree. LayerTree is about 60% smaller at $D = 16$; a radix-4 tree needs 2,031 vB, and an Ark member of a 256-member round unrolls 1,311 vB. Under mass exit a tree's interior is shared, about 261 vB per member at radix 2 and 191 vB at radix 4 with the claim, against 161 vB for a cohort leaving by $\mathsf{REWRITE}_6$. The model prices neither the latency of $D$ sequential confirmations nor the larger opcode set.

*Artefact.* Every number here is regenerated from the repository [25] at the revision on the title page by the test suites and generators under `tools/` (1,054

FULL, 268 MID, 55 MIN, 972 CCV and 210 channel checks), by `go test` and by `lake build`.

## 11 Related Work

*Payment pools.* CoinPool [21] is a shared UTXO with per-user balances whose one-transaction withdrawal rests on a new opcode, `OP_MERKLESUB`; `OP_TAPLEAF_UPDATE_VERIFY` [37] recreates a pool output with the leaver's key and leaf removed, and `OP_EVICT` [42] removes unresponsive members. LayerTree reaches one-transaction exit from opcodes proposed as general primitives, holds balances in a Merkle-sum tree rather than keys in a taptree, and adds lease-based eviction, cohort grafts, channel leaves and a composable statement. *Transaction trees.* Channel factories [6] need all $N$ members to sign every change. Ark [17], deployed today [1], batches users into rounds whose outputs are leaves of a pre-signed MuSig2 tree; covenant Ark replaces the signing with CTV templates [30] but keeps the tree, its unrolled exit and its expiry. Timeout trees [18] and SuperScalar [43] add laddered expiries and Decker-Wattenhofer factories; MATT-style trees [14,15] replace signatures by templates. *Beyond Bitcoin.* Plasma's forced exits [28] prove a balance against a committed root, enforced by a contract. Proofs of liabilities [16,8] use Merkle-sum trees, where a node that hashes only its children's total admits the inflation that hashing each child's sum avoids. *Components.* The Merkle-sum sparse Merkle tree is Taproot Assets' [23], truncated and verified in script; the data-tweaked key follows BIP-443 [15]; the generator-key signature is Poelstra's [27], and its ECDSA analogue has been used on Bitcoin Cash as `OP_PUSH_TX`. Liquid's tapscript [5] has `OP_CAT`, CSFS, per-field introspection and 64-bit arithmetic, so MID is close to expressible there; Simplicity [22] would express the predicates directly.

## 12 Discussion and Conclusion

All activity is serialised through one output, so an honest exit's delay is set by policy and fees, which [C] names rather than derives; cluster mempool [9] raises $\kappa$ for a cooperative tree but shares the cluster with every descendant anyone attaches, and fee pinning is outside [C]. Two open changes would narrow the dependence: a relative lock on transitions that remove no value, so that only removals chain, and an exit authorised by a CSFS signature over $m$, the leaf and the payout rather than the outpoint, so a tower could rebroadcast it. A Liquid prototype of MID [5] is the obvious next experiment.

LayerTree specifies a shared output by what it lets members do (join, leave alone, be evicted after a lease, be swept as a cohort) and realises it with a tree kept off-chain as data: a one-transaction exit of about a kilobyte, no pre-signing, no expiry of the shared output, from proposed opcodes; its price is that all activity funnels through one UTXO. Vouchers pay members off-chain against that output. Future work: a verified compiler, a proof that channel leaves compose, and a P model [12].

## References


1. Ark Labs: arkd: Backend services powering Arkade. Repository, https://github.com/arkade-os/arkd (2023)
2. Badertscher, C., Maurer, U., Tschudi, D., Zikas, V.: Bitcoin as a transaction ledger: A composable treatment. In: CRYPTO 2017. LNCS, vol. 10401, pp. 324–356. Springer (2017)
3. Bitcoin Core: Mempool policy defaults: Ancestor and descendant count limits (25) and cluster limit (64). src/policy/policy.h at v31.1, https://github.com/bitcoin/bitcoin/blob/v31.1/src/policy/policy.h (2026)
4. Black, B., Rubin, J.: BIP-348: CHECKSIGFROMSTACK. https://github.com/bitcoin/bips/blob/master/bip-0348.md (2024)
5. Blockstream: Elements tapscript opcodes: Introspection, streaming SHA256, 64-bit arithmetic, ECMULSCALARVERIFY, TWEAKVERIFY. https://github.com/ElementsProject/elements/blob/master/doc/tapscript_opcodes.md (2021)
6. Burchert, C., Decker, C., Wattenhofer, R.: Scalable funding of bitcoin micropayment channel networks. In: Royal Society Open Science. vol. 5 (2018)
7. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: FOCS 2001. pp. 136–145. IEEE (2001)
8. Chalkias, K., Chatzigiannis, P., Ji, Y.: Broken proofs of solvency in blockchain custodial wallets and exchanges. In: Financial Cryptography and Data Security, FC 2022 International Workshops. LNCS, Springer (2022)
9. Daftuar, S.: An overview of the cluster mempool proposal. Delving Bitcoin, https://delvingbitcoin.org/t/an-overview-of-the-cluster-mempool-proposal/393 (2024)
10. Decker, C., Russell, R., Osuntokun, O.: eltoo: A simple layer2 protocol for bitcoin. https://blockstream.com/eltoo.pdf (2018)
11. Decker, C., Towns, A.: BIP-118: SIGHASH_ANYPREVOUT for taproot scripts. https://github.com/bitcoin/bips/blob/master/bip-0118.mediawiki (2017)
12. Desai, A., Gupta, V., Jackson, E., Qadeer, S., Rajamani, S., Zufferey, D.: P: Safe asynchronous event-driven programming. In: PLDI 2013. pp. 321–332. ACM (2013)
13. Heilman, E., Sabouri, A.: BIP-347: OP_CAT in tapscript. https://github.com/bitcoin/bips/blob/master/bip-0347.mediawiki (2023)
14. Ingala, S.: Merkleize all the things. bitcoin-dev mailing list, https://merkle.fun (2022)
15. Ingala, S.: BIP-443: OP_CHECKCONTRACTVERIFY. https://github.com/bitcoin/bips/blob/master/bip-0443.mediawiki (2025)
16. Ji, Y., Chalkias, K.: Generalized proof of liabilities. In: ACM CCS 2021. ACM (2021)
17. Keceli, B.: Ark: An alternative privacy-preserving second layer solution. bitcoin-dev mailing list, 22 May 2023, https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2023-May/021694.html (2023)
18. Law, J.: Scaling lightning with simple covenants. bitcoin-dev mailing list, https://github.com/JohnLaw2/ln-scaling-covenants (2023)
19. Merkle, R.C.: A digital signature based on a conventional encryption function. In: CRYPTO 1987. LNCS, vol. 293, pp. 369–378. Springer (1988)
20. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: CADE 2021. LNCS, vol. 12699, pp. 625–635. Springer (2021)
21. Naumenko, G., Riard, A.: CoinPool: Efficient off-chain payment pools for Bitcoin. https://github.com/coinpool-dev/paper/blob/master/coinpool-

v0.1.pdf; OP_MERKLESUB draft BIP, https://github.com/ariard/bips/blob/coinpool-bips/bip-merklesub.mediawiki (2022)
22. O'Connor, R.: Simplicity: A new language for blockchains. PLAS 2017, https://blockstream.com/simplicity.pdf (2017)
23. Osuntokun, O.: Merkle sum sparse Merkle trees. Draft BIP (Taproot Assets), https://github.com/Roasbeef/bips/blob/bip-tap/bip-tap-ms-smt.mediawiki (2021)
24. Osuntokun, O.: Elliptic curve operations for Bitcoin script. Draft BIP, bitcoin/bips PR #1945 (closed unmerged), https://github.com/bitcoin/bips/pull/1945 (2025)
25. Osuntokun, O.: LayerTree: Specification, reference implementation and formal model. https://github.com/roasbeef/layertree (2026)
26. Osuntokun, O.: Simple taproot channels (extension BOLT, feature bits 80/81). lightning/bolts PR #995, merged May 2026, https://github.com/lightning/bolts/blob/master/bolt-simple-taproot.md (2026)
27. Poelstra, A.: CAT and Schnorr tricks I and II. Blog posts, 20 Jan and 10 Feb 2021, https://www.wpsoftware.net/andrew/blog/cat-and-schnorr-tricks-i.html, https://www.wpsoftware.net/andrew/blog/cat-and-schnorr-tricks-ii.html (2021)
28. Poon, J., Buterin, V.: Plasma: Scalable autonomous smart contracts. White paper, https://plasma.io/plasma.pdf (2017)
29. Poon, J., Dryja, T.: The bitcoin lightning network: Scalable off-chain instant payments. https://lightning.network/lightning-network-paper.pdf (2016)
30. Roose, S.: The Ark case for CTV. Delving Bitcoin, https://delvingbitcoin.org/t/the-ark-case-for-ctv/1528 (2025)
31. Rubin, J.: BIP-119: CHECKTEMPLATEVERIFY. https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki (2020)
32. Rubin, J.: BIP-449: OP_TWEAKADD, x-only key tweak addition. https://github.com/bitcoin/bips/blob/master/bip-0449.md (2026)
33. Russell, R.: OP_TX. Draft BIP on a personal fork, https://github.com/rustyrussell/bips/blob/guilt/varops/bip-unknown-optx.mediawiki (2024)
34. Russell, R., Moik, J.: BIP-440: Varops budget for script runtime constraint. https://github.com/bitcoin/bips/blob/master/bip-0440.mediawiki (2025)
35. Russell, R., Moik, J.: BIP-441: Restoration of disabled script (tapleaf 0xC2). https://github.com/bitcoin/bips/blob/master/bip-0441.mediawiki (2025)
36. Sanders, G.: LN-Symmetry (eltoo) transaction formats, draft. https://github.com/instagibbs/bolts/blob/eltoo_draft/XX-eltoo-transactions.md (2023)
37. Towns, A.: TAPLEAF_UPDATE_VERIFY covenant opcode. bitcoin-dev mailing list, 9 Sep 2021, https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2021-September/019419.html (2021)
38. Wuille, P., Nick, J., Ruffing, T.: BIP-340: Schnorr signatures for secp256k1. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki (2020)
39. Wuille, P., Nick, J., Towns, A.: BIP-341: Taproot: SegWit version 1 spending rules. https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki (2020)
40. Wuille, P., Nick, J., Towns, A.: BIP-342: Validation of taproot scripts. https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki (2020)
41. Zhao, G.: BIP-431: Topology restrictions for pinning. https://github.com/bitcoin/bips/blob/master/bip-0431.mediawiki (2024)
42. ZmnSCPxj: OP_EVICT: An alternative to OP_TAPLEAFUPDATEVERIFY. bitcoin-dev mailing list, 18 Feb 2022, https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2022-February/019926.html (2022)

**Table 3.** Deployment assumptions (spec 4.1, `spec/layertree.md`). A proposal *defines* its opcode for one tapleaf version; a tier may *assume* it in another.

| | assumption | tiers |
|---|---|---|
| [A-1] | `OP_TX` [33] (a draft on a personal fork, defined for `0xC2` with BIP-440 costs) pushes amounts as 8-byte little-endian strings, which FULL compares with BIP-441 numerics | FULL, MID, CCV |
| [A-2] | CSFS (BIP-348, defined for `0xC0`) available in `0xC2` at the cost of one signature check | FULL, CCV |
| [A-3] | the EC-ops draft [24] available in `0xC2` at its stated costs | FULL |
| [A-4] | opcode numbers: `OP_TX` and `OP_EC_POINT_NEGATE` both claim `0xbd`, BIP-443 and `OP_EC_POINT_ADD` both `0xbb`; the simulator parks `OP_TX`, `OP_TWEAKADD` and CCV at `0xbf`, `0xc0`, `0xc1`, so the golden scripts are byte-exact under that numbering only (sizes are unaffected) | all |
| [A-5] | `OP_TX` in `0xC0` with `CScriptNum` outputs, which no proposal defines | MID, CCV |
| [A-6] | BIP-443 (defined for `0xC0`) also in `0xC2`; its sigop cost, open upstream, charged as one signature check; amount rules as of `bitcoin/bips` commit `24e96e87`. Conservation rests on the tier's `OP_TX` equalities, so a change to BIP-443 can only make honest spends fail | CCV |
| [MIN] | `OP_CHECKSIG` under key $G$ with $s = e + 1$ accepts only if the script-built message is the real one, barring a collision in the TapSighash or challenge hash | MIN |
| [C] | contention: $\kappa$ and $\beta$ of §6 (policy and fees, not consensus) | all |

43. ZmnSCPxj: SuperScalar: Laddered timeout-tree-structured Decker-Wattenhofer factories. Delving Bitcoin, `https://delvingbitcoin.org/t/superscalar-laddered-timeout-tree-structured-decker-wattenhofer-factories/1143` (2024)

# A Assumptions and Measured Sizes

Table 3 lists the deployment assumptions of §7, and Table 4 the script, witness and transaction sizes behind Figure 3 and Table 2.

**Table 4.** Measured $\mathsf{EXIT}_1$ cost (FULL unless noted) on the model of §10. Witness weight is script, witness data and a 129-byte control block for the eight-leaf reference script set (each doubling of the set adds 8 vB); the transaction adds the 946 WU skeleton. The sparse witness is $244 + 41k$ bytes for $k$ non-empty siblings: $k = 2$ on the measured six-leaf tree (326 B), and $k \approx \log_2 N$, so the rows give the estimate at $N = 2^{16}$ ($k = 16$). Last column: unamortised radix-2 transaction tree of $2^D$ leaves, $150D + 111$ vB; sparse $D$ is address width.

| profile | $D$ | script (B) | witness data (B) | witness (WU) | tx (vB) | radix-2 tx tree (vB) |
|---|---|---|---|---|---|---|
| dense | 8 | 1,397 | 574 | 2,100 | 762 | 1,311 |
| dense | 12 | 1,749 | 738 | 2,616 | 891 | 1,911 |
| dense | 16 | 2,101 | 902 | 3,132 | 1,020 | 2,511 |
| dense | 20 | 2,453 | 1,066 | 3,648 | 1,149 | 3,111 |
| sparse | 32 | 5,147 | $\approx$ 900 | $\approx$ 6,176 | $\approx$ 1,781 | n/a |
| sparse | 64 | 9,499 | $\approx$ 900 | $\approx$ 10,528 | $\approx$ 2,869 | n/a |
| MID dense | 16 | 3,367 | 978 | 4,474 | 1,355 | |
| MIN dense | 16 | 3,640 | 1,175 | 4,944 | 1,473 | |
| CCV dense | 16 | 1,716 | 838 | 2,683 | 908 | |